

Exercise – 1 Singleton Pattern:

Main.java:

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Logger logger1 = Logger.getInstance();  
  
        Logger logger2 = Logger.getInstance();  
  
  
        logger1.log("First log message");  
  
        logger2.log("Second log message");  
  
  
        // Prove they're the same instance  
  
        System.out.println("Same instance? " + (logger1 == logger2));  
  
        System.out.println("logger1 hashCode: " + logger1.hashCode());  
  
        System.out.println("logger2 hashCode: " + logger2.hashCode());  
  
    }  
}
```

Logger.java:

```
import java.time.LocalDateTime;  
  
  
  
  
  
  
  
  
  
public class Logger {  
  
  
  
  
  
  
  
  
  
    // Single instance, volatile so changes are visible across threads  
  
    private static volatile Logger instance;  
  
  
  
  
  
  
  
  
  
    // Private constructor to prevent instantiation from outside
```

```

private Logger() {

    System.out.println("Logger instance created.");

}

// Thread-safe accessor

public static Logger getInstance() {

    if (instance == null) {

        synchronized (Logger.class) {

            if (instance == null) {

                instance = new Logger();

            }

        }

    }

    return instance;

}

public void log(String message) {

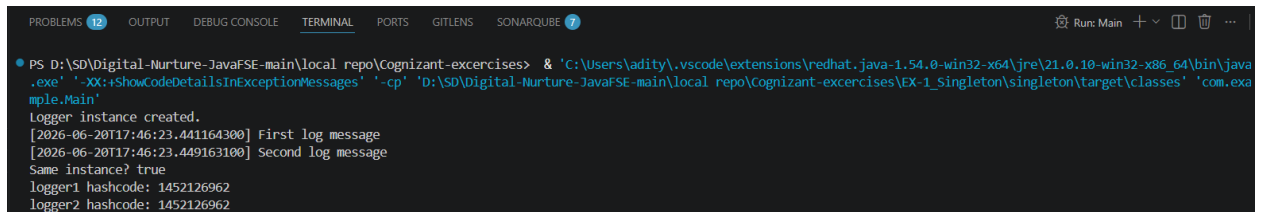
    System.out.println("[ " + LocalDateTime.now() + " ] " + message);

}

}

```

Output:



```

PS D:\SD\Digital-Nurture-JavaFSE-main\local_repo\Cognizant-exercices> & 'C:\Users\adity\.vscode\extensions\redhat.java-1.54.0-win32-x64\jre\21.0.10-win32-x86_64\bin\java
.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'D:\SD\Digital-Nurture-JavaFSE-main\local_repo\Cognizant-exercices\EX-1_Singleton\singleton\target\classes' 'com.exa
mple.Main'
Logger instance created.
[2026-06-20T17:46:23.441164300] First log message
[2026-06-20T17:46:23.449163100] Second log message
Same instance? true
logger1 hashCode: 1452126962
logger2 hashCode: 1452126962

```